NAZARBAYEV UNIVERSITY

Programming for Engineers

Week 2: Strings, Branching, Conditionals, Indentation, and Iteration

Yanwei Wang
School of Engineering
Nazarbayev University

Fall 2026



Week 2 at a glance

Lecture 4

Strings and Boolean questions

- text as a Python object;
- input, comparison, and conversion;
- conditions that evaluate to True or False.

Lecture 5

Branching and decision rules

- if, else, and elif;
- indentation and logical operators;
- validation and engineering classifications.

Lecture 6

Iteration and loops

- for, range, and while;
- counters and accumulators;
- repeated engineering calculations.

Ask a question → Choose a path → Repeat with purpose → Test the boundaries

NAZARBAYEV UNIVERSITY

LECTURE 4

Strings and Boolean Questions

How Python represents text, accepts input, and turns engineering questions into True or False

NAZARBAYEV UNIVERSITY

Lecture 4 learning goals

By the end of this lecture, you should be able to:

1. explain what a Python string object represents;
2. distinguish text such as `"12"` from the number `12`;
3. use simple string operations for labels, prompts, and reports;
4. explain why `input()` returns a string;
5. create Boolean expressions using comparison operators;
6. distinguish assignment `=` from equality testing `==`;
7. read a condition as a precise engineering question.

The transition this week

Week 1 taught Python to calculate. Week 2 begins teaching Python to ask questions, choose actions, and repeat work.

NAZARBAYEV UNIVERSITY

Retrieval warm-up: predict before running

```
x = 3
x = x + 2

print(x)
print("3" + "2")
print(3 + 2)
print(type("3"))
```

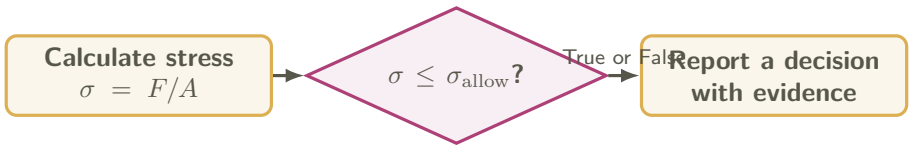
Four questions

1. What value is stored in x?
2. Which line joins text?
3. Which line adds numbers?
4. What type is "3"?

Individual - 2 min

NAZARBAYEV UNIVERSITY

From a calculation to a decision



Numbers

Store measurements and calculated quantities.

Strings and Booleans

Store labels and questions needed to communicate and decide.

What is a string object?

```
course = "ENG 101"
material = 'steel'
unit = "MPa"
message = "Design check complete"

print(course)
print(type(material))
```

A string, `str`

is an ordered sequence of characters used for:

- names and labels;
- prompts and messages;
- units and headings;
- file names and identifiers.

Quotation marks matter

`steel` is treated as a variable name. `"steel"` is text.

NAZARBAYEV UNIVERSITY

Text that looks numerical is still text

```
a = 12
b = "12"

print(type(a))
print(type(b))
print(a * 3)
print(b * 3)
```

Outputs

```
<class 'int'>
<class 'str'>
36
121212
```

Key idea

Appearance does not determine type. The object type determines which operations make sense.

NAZARBAYEV UNIVERSITY

Operators can mean different things for strings

```
first = "Engineering"
second = "Computation"

course_name = first + " " + second
separator = "-" * 28

print(separator)
print(course_name)
print(separator)
```

Overloaded operators

- For numbers, + means addition.
- For strings, + means concatenation.
- For a string and integer, * means repetition.

Not every combination is meaningful

`"A" * "B"` raises a `TypeError`.

NAZARBAYEV UNIVERSITY

A string has length and positions

```
code = "PUMP-07A"

print(len(code))
print(code[0])
print(code[-1])
```

Interpretation

- `len(code)` counts characters;
- index 0 means the first character;
- index `-1` means the last character.

Preview

Week 3 will develop string indexing, slicing, and manipulation in detail.

Zero-based indexing

A string of length n has forward indexes $0, 1, \ldots, n-1$.

NAZARBAYEV UNIVERSITY

Why does `input()` return a string?

```
voltage = input("Enter voltage in V: ")

print(voltage)
print(type(voltage))
```

User input begins as characters

The keyboard sends text. Python cannot assume whether 12 means:

- the number twelve;
- an equipment code;
- part of a date;
- a label.

input() always returns str

NAZARBAYEV UNIVERSITY

Convert only when the intended type is clear

```
voltage_v = float(
    input("Enter voltage in V: ")
)
number_of_sensors = int(
    input("Enter number of sensors: ")
)
asset_id = input("Enter asset ID: ")
```

Choose by meaning

- measured quantity → `float`;
- count → `int`;
- name or identifier → `str`.

A conversion can fail

`float("twelve")` raises a `ValueError`.

NAZARBAYEV UNIVERSITY

Small normalization step for text input

```
unit = input("Enter pressure unit: ")
unit = unit.strip().lower()

print(unit)
```

What the methods do

- `strip()` removes surrounding spaces;
- `lower()` converts letters to lowercase.

Why engineers care

Inputs such as `" kPa "`, `"KPA"`, and `"kpa"` can be compared consistently after normalization.

String methods are introduced here only for input normalization; Week 3 develops string manipulation more fully.

NAZARBAYEV UNIVERSITY

Boolean values answer yes-or-no questions

The type `bool`

has exactly two values:

`True` `False`

Engineering questions

- Is stress within the allowable limit?
- Is the sensor connected?
- Is the temperature above the alarm threshold?
- Is the entered unit recognized?

A condition is not vague

A useful Boolean expression must encode a precise rule whose result is unambiguously true or false.

Comparison operators create Boolean results

Operator	Meaning	Engineering example
`<`	less than	`temperature_c < 80`
`<=`	less than or equal to	`stress_mpa <= allowable_mpa`
`>`	greater than	`speed_mps > 30`
`>=`	greater than or equal to	`factor_of_safety >= 2.0`
`==`	equal to	`mode == "auto"`
`!=`	not equal to	`material != "steel"`

Read the expression as a question

`stress_mpa <= allowable_mpa` means: "Is the calculated stress no greater than the allowable stress?"

NAZARBAYEV UNIVERSITY

Assignment and equality are different operations

Assignment

```
mode = "automatic"
```

Bind the name `mode` to a string object.

Equality test

```
mode == "automatic"
```

Ask whether the current value equals the given string.

A common error

`if mode = "automatic":` is invalid. A condition uses `==`; assignment uses `=`.

NAZARBAYEV UNIVERSITY

Boolean expressions as engineering questions

```
stress_mpa = 120.0
allowable_mpa = 125.0
temperature_c = 82.0
unit = "kpa"

print(stress_mpa <= allowable_mpa)
print(temperature_c > 80.0)
print(unit == "kpa")
print(unit != "pa")
```

Predict in order

1. stress within limit?
2. temperature above limit?
3. unit exactly `"kpa"`?
4. unit different from `"pa"`?

NAZARBAYEV UNIVERSITY

Boolean expressions as engineering questions

```
stress_mpa = 120.0
allowable_mpa = 125.0
temperature_c = 82.0
unit = "kpa"

print(stress_mpa <= allowable_mpa)
print(temperature_c > 80.0)
print(unit == "kpa")
print(unit != "pa")
```

Predict in order

1. stress within limit?
2. temperature above limit?
3. unit exactly `"kpa"`?
4. unit different from `"pa"`?

Answer

`True, True, True, True`

Intervals and chained comparisons

```
temperature_c = 65.0

within_range = 20.0 <= temperature_c <= 80.0
print(within_range)
```

This closely matches the mathematical statement

$$20 \leq T \leq 80.$$

Equivalent expanded form

```
within_range = (
    temperature_c >= 20.0
    and temperature_c <= 80.0
)
```

Boundary question

Are exactly 20 °C and exactly 80 °C acceptable? The use of <= says yes.

Strings can also be compared

```
unit = "KPA"
normalized_unit = unit.strip().lower()

is_kpa = normalized_unit == "kpa"
is_recognized = (
    normalized_unit == "pa"
    or normalized_unit == "kpa"
    or normalized_unit == "mpa"
)

print(is_kpa)
print(is_recognized)
```

Two useful questions

- Is the normalized unit exactly `"kpa"`?
- Does it match any recognized unit string?

Case-sensitive by default

`"KPA" == "kpa"` is `False`.

NAZARBAYEV UNIVERSITY

Activity: predict, classify, explain

```
pressure_kpa = 200.0
limit_kpa = 200.0
mode = "manual"
number_text = "5"

print(pressure_kpa < limit_kpa)
print(pressure_kpa <= limit_kpa)
print(mode == "auto")
print(number_text * 2)
print(int(number_text) * 2)
```

For each line

1. predict the output;
2. name the result type;
3. explain the engineering or programming meaning.

Pairs - 4 min

NAZARBAYEV UNIVERSITY

Bug hunt: type, quotation, or comparison?

```
voltage = input("Enter voltage in V: ")
unit = kPa

is_high = voltage > 12
is_kpa = unit = "kPa"

print("is_high")
print(is_kpa)
```

Find and repair four issues

- numeric input was not converted;
- text literal lacks quotation marks;
- equality test uses the wrong symbol;
- a variable name is printed as literal text.

Pairs - 4 min

Lecture 4 recap and exit ticket

Today you connected

- strings to prompts, labels, and identifiers;
- input text to explicit type conversion;
- comparisons to Boolean engineering questions;
- boundary symbols to precise decision rules.

Exit ticket

For an acceptable temperature interval of 20 °C to 80 °C inclusive:

1. write the Boolean expression;
2. state its result at $T = 80$;
3. explain why = would be wrong.

Individual - 3 min

NAZARBAYEV UNIVERSITY

LECTURE 5

Branching and Engineering Decision Rules

Using if, elif, else, indentation, and logical operators to encode transparent engineering decisions

NAZARBAYEV UNIVERSITY

Lecture 5 learning goals

By the end of this lecture, you should be able to:

1. explain how branching changes the flow of a program;
2. write and trace `if`, `if-else`, and `if-elif-else` structures;
3. use indentation to define the intended code block;
4. combine conditions with `and`, `or`, and `not`;
5. order classification rules correctly;
6. validate simple engineering inputs before calculation;
7. test representative, boundary, and invalid cases;
8. distinguish a program decision from proof of real-world safety.

NAZARBAYEV UNIVERSITY

Opening problem: from stress to a decision

Steel bracket

A bracket has:

- yield strength $\sigma_y = 250\,\text{MPa}$;
- required factor of safety $\text{FoS}_{\text{req}} = 2.0$;
- calculated stress $\sigma = 120\,\text{MPa}$.

The allowable stress is

$$\sigma_{\text{allow}} = \frac{\sigma_y}{\text{FoS}_{\text{req}}}.$$

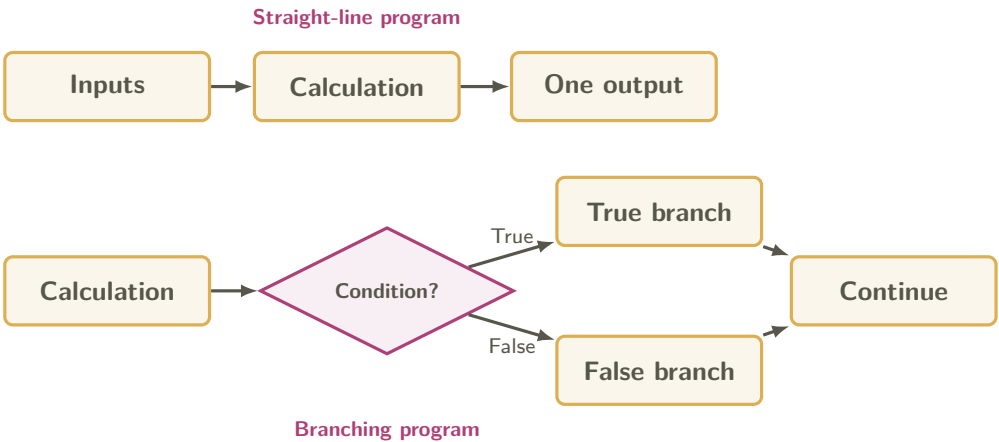
What must the program decide?

- Is the input valid?
- Does the stress pass the rule?
- Which values should be reported?
- What limitations remain?

Think - pair - share, 3 min

NAZARBAYEV UNIVERSITY

Straight-line execution versus branching



The anatomy of an `if` statement

```
factor_of_safety = 2.3

if factor_of_safety >= 2.0:
    print("Design passes the stated rule.")

print("Calculation complete.")
```

Three syntax elements

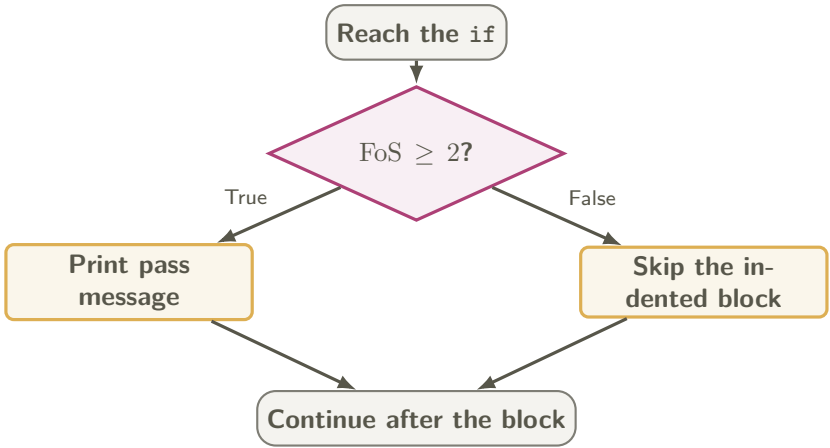
1. keyword `if`;
2. Boolean condition and colon;
3. indented block.

Control flow

The indented line runs only when the condition is `True`. The unindented final line always runs.

NAZARBAYEV UNIVERSITY

How the first branch flows



NAZARBAYEV UNIVERSITY

Indentation defines the decision block

Correct

```
temperature_c = 95.0

if temperature_c > 80.0:
    print("WARNING")
    print("Improve cooling")

print("Check complete")
```

Logic error

```
temperature_c = 60.0

if temperature_c > 80.0:
    print("WARNING")
print("Improve cooling")

print("Check complete")
```

Diagnostic question

Which messages should appear only when the temperature exceeds the limit?

NAZARBAYEV UNIVERSITY

Activity: mark the blocks before running

```
stress_mpa = 140.0
allowable_mpa = 125.0

if stress_mpa <= allowable_mpa:
    print("Stress check passed")
    print("Within allowable limit")

print("End of report")
```

Do three things

1. box the conditional block;
2. predict every printed line;
3. change one indentation level to create a misleading report.

Pairs - 4 min

NAZARBAYEV UNIVERSITY

Two alternatives with `if-else`

```
factor_of_safety = 1.6

if factor_of_safety >= 2.0:
    print("Design is acceptable")
else:
    print("Design is not acceptable")
```

Exactly one branch runs

- the `if` block runs when the condition is true;
- the `else` block runs otherwise;
- execution then continues below both blocks.

No condition follows `else`

It means "all remaining cases."

NAZARBAYEV UNIVERSITY

Live coding: transparent stress pass/fail report

```
yield_strength_mpa = 250.0
required_fos = 2.0
stress_mpa = 120.0

allowable_mpa = yield_strength_mpa / required_fos

print("STRESS CHECK")
print(f"Stress:           {stress_mpa:.2f} MPa")
print(f"Allowable stress: {allowable_mpa:.2f} MPa")

if stress_mpa <= allowable_mpa:
    print("Decision: PASS")
else:
    print("Decision: FAIL")
```

Why this is stronger

The report shows both the value and the criterion, so another engineer can inspect why the branch was taken.

Still not proof of safety

The result is conditional on the simplified model, input data, and chosen design rule.

NAZARBAYEV UNIVERSITY

Boundary values are part of the specification

Rule A

`stress_mpa < allowable_mpa`

- equality fails;
- only strictly lower stress passes.

Rule B

`stress_mpa <= allowable_mpa`

- equality passes;
- the limit is inclusive.

Test exactly at the threshold

At $\sigma = \sigma_{\text{allow}}$, the two rules produce different decisions. The code must match the intended engineering specification.

More than two outcomes with `elif`

```
factor_of_safety = 2.8

if factor_of_safety < 1.0:
    category = "failure expected"
elif factor_of_safety < 2.0:
    category = "insufficient margin"
elif factor_of_safety < 4.0:
    category = "acceptable by stated rule"
else:
    category = "conservative"

print(category)
```

Top-to-bottom search

Python selects the first condition that is `True`; later branches are skipped.

Useful for

classifications, operating states, warning levels, and quality categories.

Condition order can change the answer

Incorrect order

```
fos = 1.2

if fos < 4.0:
    category = "acceptable"
elif fos < 2.0:
    category = "unsafe"
else:
    category = "conservative"
```

Better order

```
fos = 1.2

if fos < 1.0:
    category = "failure expected"
elif fos < 2.0:
    category = "unsafe"
elif fos < 4.0:
    category = "acceptable"
else:
    category = "conservative"
```

Design principle

For increasing thresholds, test the most restrictive lower category first.

NAZARBAYEV UNIVERSITY

Logical operators combine engineering conditions

Operator	Meaning	Example decision rule
`and`	true only when both conditions are true	temperature and pressure must both be within limits
`or`	true when at least one condition is true	warn if temperature or pressure exceeds a limit
`not`	reverses a Boolean value	report an error if the sensor is not connected

Translate plain language first

Write the decision rule in words, identify each individual question, then combine the Boolean expressions.

Operating limits with `and`, `or`, and `not`

```
temperature_c = 75.0
pressure_kpa = 180.0
sensor_connected = True

temperature_ok = temperature_c <= 80.0
pressure_ok = pressure_kpa <= 200.0

if temperature_ok and pressure_ok and sensor_connected:
    print("System is within stated limits")

if not sensor_connected:
    print("ERROR: sensor disconnected")

if temperature_c > 80.0 or pressure_kpa > 200.0:
    print("WARNING: at least one limit exceeded")
```

Meaningful Boolean names

`temperature_ok` and `pressure_ok` make the rule easier to read, test, and explain.

NAZARBAYEV UNIVERSITY

A classic Boolean logic bug

```
unit = "psi"

if unit == "pa" or "kpa" or "mpa":
    print("Recognized unit")
else:
    print("Unknown unit")
```

Why it is wrong

The strings `"kpa"` and `"mpa"` are nonempty and therefore truthy. The condition does not compare `unit` to each option.

Clear correction

```
if (unit == "pa" or
    unit == "kpa" or
    unit == "mpa"):
```

Predict before running

NAZARBAYEV UNIVERSITY

Input validation protects the calculation

```
force_n = float(input("Enter force in N: "))
area_mm2 = float(input("Enter area in mm^2: "))

if area_mm2 <= 0:
    print("ERROR: area must be positive")
else:
    area_m2 = area_mm2 / 1_000_000
    stress_mpa = force_n / area_m2 / 1_000_000
    print(f"Stress = {stress_mpa:.2f} MPa")
```

Validate before use

Some values are invalid before the main formula is evaluated:

- nonpositive area;
- zero resistance;
- negative mass;
- impossible temperature.

NAZARBAYEV UNIVERSITY

Floating-point decisions usually need tolerance

```
x = 0.1 + 0.2

print(x == 0.3)
print(abs(x - 0.3) < 1e-12)
```

Ask "close enough," not always "identical"

Floating-point values are approximations. A tolerance expresses the numerical accuracy required by the task.

Tolerance is an engineering choice

1e−12 is illustrative, not universally appropriate. Choose a scale related to the problem and measurement precision.

NAZARBAYEV UNIVERSITY

Integrated example: factor-of-safety classifier

```
force_n = 10_000.0
area_mm2 = 80.0
yield_strength_mpa = 250.0

area_m2 = area_mm2 / 1_000_000
stress_mpa = force_n / area_m2 / 1_000_000
fos = yield_strength_mpa / stress_mpa

if fos < 1.0:
    category = "failure expected"
elif fos < 2.0:
    category = "insufficient margin"
elif fos < 4.0:
    category = "acceptable by stated rule"
else:
    category = "conservative"

print(f"Stress = {stress_mpa:.2f} MPa")
print(f"Factor of safety = {fos:.2f}")
print(f"Classification = {category}")
```

Expected boundary

Here $\mathrm{FoS} = 2.00$, so the program reaches the third branch.

State the scope

This is a simplified instructional rule, not a universal design standard.

NAZARBAYEV UNIVERSITY

Build evidence with a test matrix

Test type	Input	Expected branch	Why it matters
Representative	$\mathrm{FoS} = 2.8$	acceptable	normal intended use
Boundary	$\mathrm{FoS} = 2.0$	acceptable	checks inclusion at threshold
Just below	$\mathrm{FoS} = 1.999$	insufficient	checks operator and ordering
Invalid	$A = 0$	error before calculation	prevents meaningless division
Extreme	$\mathrm{FoS} = 10$	conservative	checks final `else`

A single successful run is weak evidence

Test every branch, every important boundary, and invalid input that the program is expected to handle.

NAZARBAYEV UNIVERSITY

Assignment 1 launch: decisions and repetition

Core task pattern

Develop a small engineering program that:

1. define inputs and units, then validate critical values;
2. calculate one or more engineering quantities;
3. classify the result with transparent branches;
4. repeat a related calculation and print a clear report.

Evidence to include

- plain-language algorithm and readable code;
- representative, boundary, and invalid-input tests;
- brief engineering interpretation;
- disclosure of permitted assistance.

Authoritative brief

Follow the final Moodle scope, deadline, and submission rules.

Lecture 5 recap and exit ticket

A reliable decision program

- states the rule and boundary clearly;
- uses correct comparison and Boolean logic;
- indents each block intentionally;
- validates inputs before risky operations;
- reports evidence, not only a label;
- is tested across all branches.

Exit ticket

Write a three-category pressure rule:

- below 100 kPa: low;
- 100 kPa to 200 kPa inclusive: normal;
- above 200 kPa: high.

Identify both boundary cases.

Individual - 3 min

NAZARBAYEV UNIVERSITY

LECTURE 6

Iteration and Repeated Engineering Calculations

Using for and while loops to express time evolution, repeated measurements, design searches, and stopping conditions

NAZARBAYEV UNIVERSITY

Lecture 6 learning goals

By the end of this lecture, you should be able to:

1. explain why engineering programs need repetition;
2. write and trace a `for` loop over `range()`;
3. explain why the stop value of `range()` is excluded;
4. distinguish code inside and outside a loop body;
5. use counters, accumulators, and running values;
6. write a `while` loop using initialization, condition, and update;
7. identify an infinite loop and explain how the loop makes progress;
8. choose between `for` and `while`;
9. test a loop's boundaries and stopping condition.

NAZARBAYEV UNIVERSITY

Opening problem: one formula, many times

Dropped object

A ball is dropped from height $h_0 = 100\,\text{m}$. Ignoring air resistance,

$$h(t) = h_0 - \frac{1}{2}gt^2, \qquad g = 9.81\,\text{m/s}^2.$$

Print the height at each integer second and stop after the ball reaches the ground.

Why one calculation is not enough

We need the same model at many values of t, and we may not initially know the exact stopping time.

Sketch an algorithm - 3 min

Why engineers need loops

Across time

- cooling;
- charging;
- tank filling;
- motion.

Across cases

- load levels;
- trial dimensions;
- operating points;
- cost alternatives.

Across data

- characters;
- sensor readings;
- samples;
- records.

The recurring structure

Repeat a defined block, change one or more values, and stop according to a clear rule.

NAZARBAYEV UNIVERSITY

Manual repetition does not scale

Repeated lines

```
h0 = 100.0
g = 9.81

h0_s = h0 - 0.5 * g * 0**2
h1_s = h0 - 0.5 * g * 1**2
h2_s = h0 - 0.5 * g * 2**2
h3_s = h0 - 0.5 * g * 3**2
```

Repeated structure

```
h0 = 100.0
g = 9.81

for time_s in range(0, 4):
    height_m = h0 - 0.5 * g * time_s**2
    print(time_s, height_m)
```

Why the loop is better

It is shorter, easier to modify, easier to inspect, and less likely to contain copy-and-paste errors.

Your first `for` loop

```
for time_s in range(0, 5):
    print(time_s)

print("Loop finished")
```

Read it aloud

For each value of `time_s` from 0 up to, but not including, 5, print the value.

Output sequence

`0, 1, 2, 3, 4`, followed once by `Loop finished`.

NAZARBAYEV UNIVERSITY

Trace a loop one pass at a time

Pass	Value of `time_s`	Action
1	0	print 0
2	1	print 1
3	2	print 2
4	3	print 3
5	4	print 4

Hand simulation

Pretend to be the interpreter: record the loop variable and changed values on each pass. A short trace table is often the fastest way to explain surprising output.

NAZARBAYEV UNIVERSITY

Three common forms of `range()`

Form	Meaning	Generated values
`range(5)`	start at 0; stop before 5	0, 1, 2, 3, 4
`range(2, 7)`	start at 2; stop before 7	2, 3, 4, 5, 6
`range(0, 11, 2)`	start at 0; increase by 2; stop before 11	0, 2, 4, 6, 8, 10
`range(5, 0, -1)`	start at 5; decrease by 1; stop above 0	5, 4, 3, 2, 1

The stop value is excluded

To include $t = 10$ in an integer table starting at 0, use `range(0, 11)`.

NAZARBAYEV UNIVERSITY

Activity: predict the sequence and number of passes

```
for i in range(4):
    print(i)

for j in range(1, 6):
    print(j)

for k in range(10, 1, -3):
    print(k)
```

For each loop

1. list the values;
2. count the loop passes;
3. identify the excluded stop value.

Individual then pair - 4 min

NAZARBAYEV UNIVERSITY

Indentation defines the loop body

Repeated together

```
for time_s in range(3):
    print("Inside loop")
    print(time_s)

print("After loop")
```

Only one line repeats

```
for time_s in range(3):
    print("Inside loop")
print(time_s)

print("After loop")
```

Ask before running

Which statements run once per pass? Which statements run once after the loop?

NAZARBAYEV UNIVERSITY

Engineering table: velocity over time

```
acceleration_mps2 = 2.0

print("Time (s)   Velocity (m/s)")
print("------------------------")

for time_s in range(0, 11):
    velocity_mps = acceleration_mps2 * time_s
    print(f"{time_s:8d}   {velocity_mps:14.2f}")
```

The loop variable is an input

On each pass, `time_s` takes a new value and the same model is evaluated.

Trend check

With constant positive acceleration from rest, velocity should increase linearly with time.

A decision can live inside a loop

```
h0_m = 100.0
g_mps2 = 9.81

for time_s in range(0, 6):
    height_m = h0_m - 0.5 * g_mps2 * time_s**2

    if height_m > 0:
        status = "above ground"
    else:
        status = "ground reached"

    print(f"t={time_s:2d} s, h={height_m:7.2f} m, {
     status}")
```

Nested structure

The `if-else` block runs once during every loop pass.

Model limitation

The discrete table may step from a positive to a negative height; the true impact occurs between samples.

NAZARBAYEV UNIVERSITY

Counters, accumulators, and running values

Counter

Counts events or cases.

```
count += 1
```

Accumulator

Builds a running total.

```
total += value
```

Running state

Updates a system quantity.

```
volume += rate * dt
```

Initialization matters

The variable must normally be assigned an appropriate starting value before the loop begins.

NAZARBAYEV UNIVERSITY

Accumulator example: total energy over intervals

```
power_kw = 1.5
interval_h = 1.0
total_energy_kwh = 0.0

for hour in range(1, 6):
    energy_this_interval = power_kw * interval_h
    total_energy_kwh += energy_this_interval
    print(hour, total_energy_kwh)
```

Trace the accumulator

It begins at `0.0` and stores all energy added so far.

Common bug

Initializing `total_energy_kwh = 0.0` inside the loop would reset the total every pass.

Strings are iterable too

```
sample_code = "SAMPLE-2026-A7"
digit_count = 0

for character in sample_code:
    if character.isdigit():
        digit_count += 1

print(f"Number of digits = {digit_count}")
```

One character per pass

The loop variable `character` is bound successively to each character in the string.

Integration of Week 2 ideas

A loop repeats; a string supplies values; a conditional decides whether to increment the counter.

NAZARBAYEV UNIVERSITY

When the number of repetitions is not known

Use `for` when

- a sequence is known;
- the number of cases is predetermined;
- you are iterating over a string or collection;
- a table uses defined input values.

Use `while` when

- repetition continues until an event;
- the required number of passes is unknown;
- an error must fall below a tolerance;
- a state must reach a threshold.

The anatomy of a `while` loop

```
time_s = 0                    # initialization

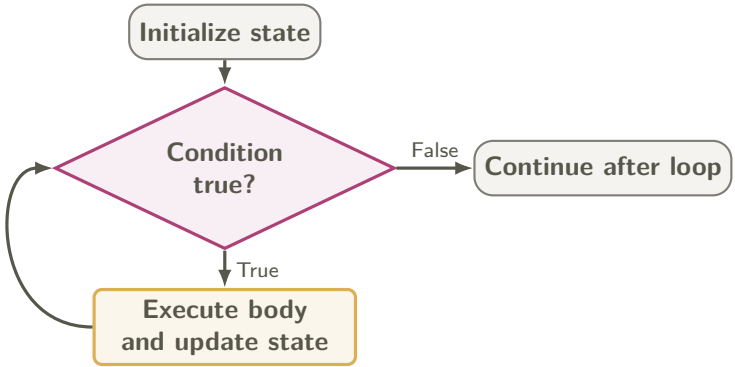
while time_s <= 10:           # condition
    print(time_s)
    time_s = time_s + 1       # update
```

Three questions

1. What is the initial state?
2. What keeps the loop running?
3. What changes so the condition eventually becomes false?

Initialize - test - update

How a `while` loop flows



The condition is tested before every pass

A `while` loop can execute zero times if the condition is already false.

NAZARBAYEV UNIVERSITY

Tank filling: a state evolves until a threshold

```
capacity_l = 100.0
flow_rate_lpm = 8.0
volume_l = 0.0
time_min = 0.0
dt_min = 0.1

while volume_l < capacity_l:
    time_min += dt_min
    volume_l += flow_rate_lpm * dt_min

print(f"Capacity reached at about {time_min:.1f} min")
print(f"Calculated volume = {volume_l:.2f} L")
```

Progress variable

`volume_l` increases every pass, moving toward the stopping threshold.

Approximation

The reported time depends on the step size `dt_min`. A smaller step gives a finer estimate but needs more passes.

NAZARBAYEV UNIVERSITY

Infinite loop: the condition never changes

```
time_s = 0

while time_s <= 10:
    print(time_s)
    # missing update
```

What happens?

`time_s` remains 0, so the condition stays true forever.

Debugging checklist

- Which variable controls the condition?
- Where is it initialized?
- Where is it updated?
- Does the update move toward termination?

`break` and `continue`

`break`

Exit the nearest loop immediately.

```
for area_mm2 in range(100, 1001, 50):
    stress_mpa = 8000 / area_mm2
    if stress_mpa <= 50:
        print(area_mm2)
        break
```

`continue`

Skip the rest of the current pass.

```
for area_mm2 in range(0, 501, 100):
    if area_mm2 == 0:
        continue
    stress_mpa = 5000 / area_mm2
    print(area_mm2, stress_mpa)
```

Use carefully

These statements can be clear in small searches, but excessive jumps make control flow harder to reason about.

NAZARBAYEV UNIVERSITY

Design search: first tested area that passes

```
force_n = 8000.0
allowable_stress_pa = 50e6
required_area_mm2 = None

for area_mm2 in range(100, 1001, 50):
    area_m2 = area_mm2 / 1_000_000
    stress_pa = force_n / area_m2

    if stress_pa <= allowable_stress_pa:
        required_area_mm2 = area_mm2
        break

if required_area_mm2 is None:
    print("No tested area passed")
else:
    print(f"Minimum area = {required_area_mm2} mm^2")
```

Algorithm

Test candidates from smallest to largest and stop at the first acceptable case.

What the result actually means

It is the smallest area in the tested discrete set, not necessarily the exact mathematical minimum or a complete design.

A loop needs a defensible stopping rule

Examples

- all scheduled times processed;
- tank volume reaches capacity;
- temperature falls below a target;
- error falls below a tolerance;
- first safe design is found.

Ask four questions

1. Is termination guaranteed?
2. Is the boundary inclusive or exclusive?
3. Could a step jump over the event?
4. Should there be a maximum-pass limit?

Loop design checklist

Before coding

1. What repeats?
2. Which values change?
3. What stays fixed?
4. Is a `for` or `while` loop clearer?
5. What is the stopping condition?

Before trusting

1. trace a small case by hand;
2. test zero, one, and several passes;
3. inspect the last included value;
4. test failure to find a result;
5. interpret approximation and step size.

NAZARBAYEV UNIVERSITY

Laboratory 1: branching and loops

Suggested laboratory workflow

1. begin with a supplied or partially completed program;
2. identify inputs, units, decisions, and repeated steps;
3. predict outputs before running;
4. repair syntax, indentation, and logic errors;
5. modify one requirement;
6. test and explain the final result.

Possible contexts

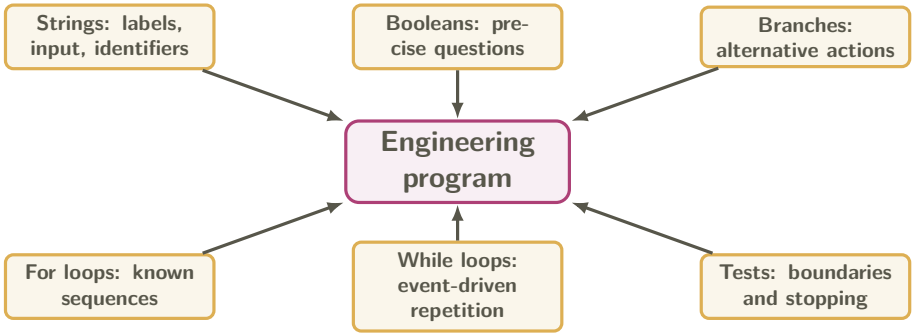
dropped object, tank filling, cooling, stress-based area search, equipment-ID validation, or operating-limit monitoring.

Individual accountability

You should be able to trace, explain, and modify every submitted line. Follow the final laboratory instructions on Moodle.

NAZARBAYEV UNIVERSITY

Week 2 concept map



Combined capability

Receive text, ask precise questions, choose a path, repeat calculations, and stop by a stated rule.

NAZARBAYEV UNIVERSITY

Week 2 exit ticket and preparation for Week 3

Exit ticket

For the loop below, state:

1. the generated values;
2. the number of passes;
3. the final value of `total`;
4. one boundary test.

```
total = 0
for i in range(1, 6):
    total += i
```

Before Week 3

- complete Assignment 1 and Lab 1 preparation;
- trace at least three loops by hand;
- test every branch boundary;
- preview string manipulation, guess-and-check, approximation, and bisection.

Individual - 3 min

Week 2 quick reference

Decisions

`==, !=`	equality and inequality tests
`<, <=, >, >=`	numerical comparisons
`and, or, not`	combine or reverse conditions
`if condition:`	execute a block when true
`elif / else`	alternative branches

Repetition

`for x in range(...)`	repeat over known values
`while condition:`	repeat while a test is true
`total += value`	accumulator update
`count += 1`	counter update
`break / continue`	exit or skip a loop pass

The engineering questions

Which rule and boundary are encoded? What changes each pass, and why must the loop stop?

Suggested reading and source basis

- Nazarbayev University, *ENG 101 - Programming for Engineers: Course Specification Report*, Fall 2026, Session 2.
- Y. Wang, *Programming for Engineers with Python*, working draft, August 2026, Chapters 2–4 and instructor notes.
- J. V. Guttag, *Introduction to Computation and Programming Using Python: With Application to Understanding Data*, 2nd ed., MIT Press, Sections 2.2–2.4.

Primary preparation for students

Run each short example, predict before execution, trace values by hand, and test representative, boundary, and invalid cases.

NAZARBAYEV UNIVERSITY

Programs become useful when they can decide and repeat

Make the rule explicit. Make the boundary testable. Make the loop progress. Keep the engineer accountable.

ENG 101 - Programming for Engineers